

ENGEE319-26B Laboratory Session 2

Controlling Equipment via WiFi

This session explores programmatic control of a Wi-Fi-enabled smart bulb utilising a Raspberry Pi. Students will investigate device control pathways and user interface design to manipulate parameters such as colour and brightness. The primary objective is to develop practical insights applicable to the architecture and operation of smart home automation systems.

Introduction

Similar to the TP-Link smart plug examined in previous sessions, the TP-Link smart bulb connects directly to a local Wi-Fi network. While its immediate range is restricted to the local network area, its intended usage scenario involves connecting to external networks via a standard home internet connection. Wi-Fi integration allows for advanced control capabilities beyond simple binary power states, including dynamic adjustments to brightness and colour.



External control initiated from outside the local network is typically restricted by standard household network security protocols. To circumvent this, TP-Link utilises cloud-based servers. Devices operate by either polling these servers periodically or maintaining open connections to receive external commands rapidly.

This laboratory utilises either the KL130 or the newer L530E smart bulb models. The L530E features enhanced security, requiring account authentication (username and password) prior to access. Furthermore, the L530E cannot be provisioned via the older Kasa application and requires the Tapo application. Both applications connect to the identical TP-Link cloud infrastructure, and the Python libraries utilized in this session are compatible with both models (though the L530E requires explicit credential transmission within the code).

Note on Provisioning: Configuration, or "provisioning," is a critical phase in IoT deployment. It involves establishing a device's identity and network location on the local Wi-Fi. Devices moving between networks must undergo a factory reset to clear prior configurations before they can be reprovisioned.

Task 1: Configuring a KL130 or L530E smart bulb.

The smart bulb is provisioned using a dedicated mobile application.

1. Connect a mobile device to the waikato_iot network (credentials are provided on the laboratory whiteboard).
2. Install the appropriate provisioning application based on the hardware model:
 - KL130: Use the "Kasa for Mobile" application.
 - L530E: Use the "Tapo" application.

3. Authenticate within the application using the following provided credentials:
 - Username: coms0108@cs.waikato.ac.nz
 - Password: engee319
4. Establish a connection to the smart bulb. Prior laboratory use necessitates a factory reset to ensure a clean configuration state. Execute a factory reset prior to proceeding.

Factory Reset Instructions:

1. To reset without losing configuration settings: Turn the controlling light switch OFF, then flip the switch ON and OFF 5 times.
2. To reset to factory defaults (required): Turn the controlling light switch OFF, then flip the switch ON and OFF 10 times.
Note: Ensure a duration of approximately 1 second between toggles; do not switch too rapidly.
3. Utilise the mobile application to verify operational control of the bulb, confirming that colour and brightness parameters can be modified successfully.

How to reset a light bulb to factory settings

To reset the Smart Wi-Fi LED Bulbs without losing the configuration settings:

1. Turn the light switch controlling the bulb to the OFF position.
2. Flip the light switch on and off **5 times**

To reset the Smart Wi-Fi LED Bulbs to factory defaults:

1. Turn the light switch controlling the bulb to the OFF position.
2. Flip the light switch on and off **10 times**

When resetting the bulb, please do not switch the bulb on and off too quickly, the time duration should be around 1s.

Task 2: Accessing the light bulb from the command line

This task utilises the python-kasa library. Official documentation is available at:

<https://python-kasa.readthedocs.io/en/latest/smartdevice.html#kasa.SmartDevice>

1. Use the 'kasa' command line program to find all devices on the local network and obtain the IP address for your lightbulb. If you are using the L530E you will see that your device will be shown as 'failed to authenticate'. You must supply username and password when you run the program. Provisioning will have given the L530E enough data to authenticate your credentials. What information do you think will be stored on the device? Does that seem secure to you? Be ready to discuss.

```
kasa --username coms0108@cs.waikato.ac.nz --password engee319
```

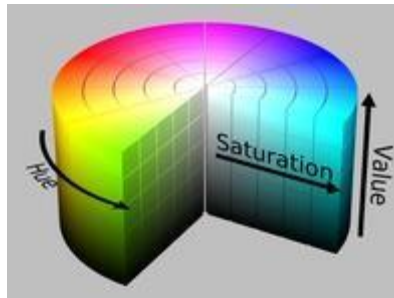
2. Determine the command syntax required to modify color and brightness via the kasa interface. On the L530E, username and password will be required with each command. Hint: kasa -help The acronym 'hsv' stands for 'hue, saturation and value' – it is used to describe the colours and intensity.

There are a number of ways of defining colours specialised for different applications: printing, television, lighting, etc. In computer graphics, the 'rgb' colour space – red/green/blue, is typically used. This roughly corresponds to the three types of colour sensor in the human eye and directly corresponding to the three colour generation components in lights and video displays. In lighting and colour television transmission, the hsv or hsl (v=value, l=luminance) system is used. The important idea here is that colour and brightness are separate dimensions (hue and value/luminance).

The lab hardware utilises the HSV (Hue, Saturation, Value) color model

- Hue (H): Represented as an angle from 0 to 360 degrees on a color cylinder (e.g., Red at 0, Blue at 240).
- Saturation (S): A scale from 0 to 100 indicating color purity. A value of 100 represents a fully saturated color, while lower values introduce white light.
- Value (V): A scale from 1 to 100 representing luminance or brightness.

Experiment with modifying H, S, and V parameters via the command line. Cross-reference graphical adjustments made in the mobile application with the numeric output from the kasa utility to formalise your understanding of the coordinate system.



Task 3: Accessing the light bulb in software (using Python)

Use the python-kasa library to operate the light bulb works in a similar way to operating the smart plug. The program lightdemo.py demonstrates some options. Experiment with the program and discuss/consider how it works. This should provide you with enough information to the task. Don't forget to change the IP address to that used by your light bulb.

lightdemo.py – showing how to operate smart light in code

```
from kasa import Discover
import asyncio

async def main():
    dev = await Discover.discover_single('10.1.1.191',
    username='coms0108@cs.waikato.ac.nz', password='engee319')
    #Print basic id information
    await dev.update()
    print (dev)

    #Turn the light on off on
    await dev.turn_on()
    await asyncio.sleep(1)
    await dev.turn_off()
```

```

await asyncio.sleep(1)
await dev.turn_on()
await asyncio.sleep(1)

#Get the 'Light' module for changing parameters
light_module = dev.modules.get ("Light")

await light_module.set_brightness(10)
await asyncio.sleep(1)
await light_module.set_brightness(50)
await asyncio.sleep(1)
await light_module.set_brightness(100)
await asyncio.sleep(1)

#Set a starting colour value
await light_module.set_hsv (0, 0, 100)
await asyncio.sleep(1)

#Display a range of colours
for hue in range (0, 360, 36):
    await light_module.set_hsv(hue, 100, 100)
    await asyncio.sleep(1)

#Set back to starting value
await light_module.set_hsv(0, 0, 100)
await asyncio.sleep(1)

#Display the current colour
await dev.update()
hsv = light_module.hsv
print("HSV: ", hsv.hue, ",", hsv.saturation, ",", hsv.value)

#Shut down tge interface to the light
await dev.disconnect()
return

asyncio.run(main())

```

We can proceed at this stage with a graphic user interface in tkinter and write a python program to control light colour (and intensity). However, the IoT home automation world commonly uses voice commands. The next few sections guide you through using Google's speech recognition and work towards audio control.

Task 4: Using audio and speech generation and recognition

Starting with audio: getting a microphone going. Whilst Bluetooth audio output works well on the Raspberry Pi, I have not had success with audio input via Bluetooth. The Raspberry Pi has an audio jack plug – but it is output only. This leaves USB as the simplest way to connect a

microphone. USB headsets work well, but it is also possible to get very small USB microphones, suitable for IOT applications.



1. Connect a USB microphone to the Raspberry Pi. Use the provided USB extension cable to distance the microphone from the acoustic interference of the device's cooling fan. It should show up as a 'PNP (Plug and Play) USB device in the audio input device list – right click on the microphone icon in the task bar (to the right) and make sure your microphone is selected.
2. The Linux system has two command line programs for recording and playing audio. To show audio input devices use the following. Make sure your microphone is connected.

```
arecord -l
```

<= that is a lower case L, for list

3. You can record a sound file (.wav) and play it back (you will need to connect a Bluetooth speaker to hear anything).

```
arecord -d 5 test.wav
```

<= record for 5 seconds to the file

```
aplay test.wav
```

<= play the file

The `-D` flag specifies input device as 'name': 'card number', 'device number'. Usually it is not needed because 'arecord' uses the default (or only) microphone.

```
arecord -D plughw:3,0 -d 5 test.wav
```

<= D for Device, d for duration

There are a number of libraries for working with audio in Python. Use 'pyaudio'

Sample audio input and output code is taken from (which compares alternative audio libraries):
<https://realpython.com/playing-and-recording-sound-python/>

The following paragraphs work through trying pyaudio and some speech applications.

1. Installing and trying pyaudio

```
sudo apt install portaudio19-dev      pip3 install  
pyaudio
```

You are supplied with a program `voice01.py` which records 5 seconds of mono audio with 16-bit samples at 16000 samples per second (preferred sampling for speech recognition) and saves to a file. The detail of the program is complicated but just check to see what sense you can make of them – in particular, how the audio is being sampled.

Try running the program and verify (using `aplay`) that audio has been recorded. Note that this program displays a large amount of diagnostic text to the screen as it runs. It is debugging information from the 'portaudio' library and can be ignored. You can prevent it displaying by using Linux command line redirection by running the program as follows:

```
python voice02.py 2>/dev/null
```

voice01.py – checking that pyaudio can get data from a microphone.

```
import pyaudio
import wave

chunk = 1024
sample_format = pyaudio.paInt16
channels = 1
fs = 16000
seconds = 5
filename = 'output3b01.wav'

#Record to memory using PyAudio
p = pyaudio.PyAudio()
print('Recording')

stream = p.open(format = sample_format,
                 #input_device_index = 3,
                 channels = channels,
                 rate = fs,
                 frames_per_buffer = chunk,
                 input = True)

frames = []

for i in range(0, int(fs / chunk * seconds)):
    data = stream.read(chunk)
    frames.append(data)

stream.stop_stream()
stream.close()
p.terminate()

print('Finished recording')

#Write the finished recording to a .wav file
wf = wave.open(filename, 'wb')
wf.setnchannels(channels)
wf.setsampwidth(p.get_sample_size(sample_format))
wf.setframerate(fs)
wf.writeframes(b''.join(frames))
wf.close()
```

Sometimes it is useful to be able to generate speech from text. The 'espeak' library can do this.

13. Install and test a text to speech application.

```
sudo apt install espeak
```

which can be tested with

```
espeak "Hello World"
```

But the more exciting possibility is speech recognition from audio. Speech recognition has become widely used and of high quality in the last two decades, mostly because collection of data on the internet has made it possible to build large collections of speech samples which have been used to train machine learning models. In IoT command applications, it is usual to specifically train a recognition engine to understand just the small vocabulary of words needed for the application.

Such recognition engines can run on the Raspberry Pi and are not very difficult to construct with freely available software. Start by using Google's online general English language transcription engine. This takes sound samples, and sends them to Google servers for translation. Conveniently Google provide a Python library to manage this process.

Speech recognition libraries are discussed here:

https://realpython.com/python-speech-recognition/?utm_source=chatgpt.com#how-speech-recognition-works-an-overview

Here are some ideas on the best speech recognition to use on Raspberry Pi:

<https://maker.pro/raspberry-pi/tutorial/the-best-voice-recognition-software-for-raspberry-pi>

Speech recognition for voice control ideas here:

<https://iotdesignpro.com/projects/speech-recognition-on-raspberry-pi-for-voice-controlled-home-automation>

1. Trying Google's speech recognition engine.

```
pip install speechrecognition
```

One more installation is needed before a demo program will work

```
sudo apt-get install flac
```

Try the following program voice02.py (adapted from one presented in the 'iotdesignpro' linked above). It should be able to translate short passages of speech. As well as displaying the translated text, it uses espeak for error messages and the recognised text. It should find the microphone automatically because it will be the default input device.

You will find the diagnostic data displayed distracting. You can avoid it by running the program with a command like the following (redirecting diagnostics to nowhere).

```
python voice02.py 2>/dev/null
```

voice02.py – access Google's speech recognition engine

```
from subprocess import call
import speech_recognition

reco = speech_recognition.Recognizer()
reco.pause_threshold = 2

def listen():
    with speech_recognition.Microphone() as source:
        reco.adjust_for_ambient_noise(source, duration=1)
```

```

    print("Say something")
    audio = reco.listen(source, timeout=5, phrase_time_limit=10)
    print("... got it")
    return audio

def recognize(audio):
    try:
        text = reco.recognize_google(audio)
        print("You said: ", text)
        return text
    except speech_recognition.UnknownValueError:
        print("Unknown value error")
        return "Failed on unknown value error"
    except speech_recognition.RequestError:
        print("Request error")
        return "Failed on request error"

while True:
    audio = listen()
    text = recognize(audio)
    call(["espeak", "-s200 -ven -z", text])
    if text == "exit":
        print("Stopping")
        exit()

```

2. Experiment with the program. How successful is it in recognising your speech? How do you find the timing of the speech entry and the response? There are four parameters in the code that affect the times taken for listening activities. What are they? Experiment to choose values that work for you. Why do you think it is not necessary to set sampling rate and precision? Be prepared to discuss.

A frequently used open source speech recognition engine is 'Sphinx', developed over a long period of time at Carnegie-Melon University. It is not under active development any more, as it has been superceded by more modern software with larger language models. It is however useful for simple language processing in IoT projects and runs locally on the machine on which it is installed. The project can be accessed at: <https://cmusphinx.github.io/>

3. Install 'pocketsphinx' as follows:

```
pip install pocketsphinx
```

Try running the program following. This has nice timing characteristics in that it monitors speech continuously and outputs recognised text reasonably quickly. You may however be disappointed by the quality of the translation. Try it and be prepared to discuss.

sphinx01.py

```

from pocketsphinx import LiveSpeech

for phrase in LiveSpeech(sampling_rate=16000):
    print(phrase)
    if str(phrase).strip() == 'stop':
        print("Stopping")

```

```
exit()
```

The default language model and dictionary that comes with the software is quite limited. It was never really intended for continuous free speech recognition. It is much more useful when provided with a limited set of phrases (called its keyphrase recognition mode). It can 'listen' to speech and catch occurrences of the phrases in its key phrase file.

For background, this web page talks about the language models that can be used with Sphinx, including the keyphrase (or key-word) system:

<https://cmusphinx.github.io/wiki/tutoriallm/>

4. The next sample tests this system. See what you can make of the following program `sphinx02.py` and its data file (`keyphrases.kws`). The numbers are the minimum recognition certainty (probability of match) at which to accept the phrase. Ideal phrases are 3 to 4 syllables. Try the program. What success do you have? Files like this can be tuned by adjusting the probability values from 1e-1 (nearly certain that it is correct) to 1e-50 (recognize anything as the phrase). Discuss.

`sphinx02.py`

```
from pocketsphinx import LiveSpeech

for speech in LiveSpeech(sampling_rate=16000,
    kws='keyphrases.kws', kws_threshold=1e-20):
    for alt in speech:
        print(alt.segments(detailed=True))
        print('Result is ', alt)
        if str(alt).strip() == 'exit':
            print("Stopping")
            exit()
```

`keyphrases.kws`

```
oh mighty computer /1e-20/
please open /1e-25/ on
/1e-3/ off /1e-3/ red /1e-
10/ blue /1e-10/ green
/1e-10/ exit /1e-10/
brighter /1e-10/ dimmer
/1e-10/ activate /1e-10/
extinguish /1e-10/
```

Task 5. Using AI

A significant issue with speech input is the lack of control over the words that a user will use, and conversely, the user doesn't know what they are allowed to say. The key phrase system of Sphinx allows the software developers to choose the allowed phrases, but then they have to be communicated to the user, who has to remember them. It is difficult to persuade consumers to undertake 'training' on the use of a product and even if they do, there will be problems of memory. If they don't use a system for a long time, they will almost certainly forget their training.

Prior to late 2022, almost no-one knew what a Large Language Model was. Two months later Chat-GPT had 100 million online users. All the large computer companies have their versions.

In this task, you will work programmatically with a large language model. The one we will use was released last year. It is called 'Gemini 2.5 Flash' and was constructed and is provided by Google. Google charges for use of Gemini, and the program requires an 'API key' for an account. The key provided for the lab is for a free trial account with limited tokens. You can also use your own Google account.

For future ref: Gemini can be accessed at: <https://aistudio.google.com/> and requires a Google account.

1. Install Google's 'Generative AI' library (with your virtual environment active).

```
pip install google-generativeai
```

Of course this didn't install very much. Just a library for sending queries to an online server.

Gemini can operate just like ChatGPT - answering questions and engaging in conversation on whatever topic you like. Your questions can include instructions on the style and level of answers you would like. The sample program I have provided `gemini01.py`, works in a slightly more structured way. It starts by providing general instructions for the conversation, that will be applied to all interactions that follow. To make it interpret its input as a colour, some instructions are written as guidance as to how it should format results. Normally, Gemini would provide explanations with its answers. In this case, you can ask it to just provide three numbers.

`gemini01.py` – access Google's generative AI system

```
from google import generativeai
from google.generativeai import types

# My general instructions to the AI model - this can be setting tone,
# style, etc. In this case I am telling it how to interpret the input
# and format the output. Note the use of triple quotes to allow a
# multi-line string

my_instructions = """
    I have a lamp which can be set by giving red green and light levels
    as numbers between 0 and 255, where 0 is dark and 255 is brightest.
    I will write descriptions of colours and you should respond in the
    format \"100,100,100\" substituting appropriate values for
    the numbers in my format. Don't add any extra text
    """

client =
generativeai.Client(api_key="AIzaSyBzV_CqzrgUy024cNdCW0Um3RPMTdwarZk")
model = "gemini-2.5-flash"
config = types.GenerateContentConfig(
    temperature=0,
    thinking_config = types.ThinkingConfig(thinking_budget=-1),
    safety_settings=[
        types.SafetySetting(
            category="HARM_CATEGORY_HARASSMENT",
            threshold="BLOCK_LOW_AND_ABOVE", # Block most
```

```

    ),
    types.SafetySetting(
        category="HARM_CATEGORY_HATE_SPEECH",
        threshold="BLOCK_LOW_AND_ABOVE", # Block most
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
        threshold="BLOCK_LOW_AND_ABOVE", # Block most
    ),
    types.SafetySetting(
        category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
        threshold="BLOCK_LOW_AND_ABOVE", # Block most
    )
],
tools=[types.Tool(googleSearch=types.GoogleSearch())],

system_instruction=[types.Part.from_text(text=my_instructions)])

while True:
    my_input_text=input("You: ")
    if my_input_text == "exit":
        break

    contents = [types.Content (role="user",
                               parts=[types.Part.from_text(text=my_input_text)])]
    response = client.models.generate_content(model=model,
                                              contents=contents, config=config)

    print("Gemini: ", response.text)

```

In fact the length of the code makes it cumbersome to work with. Much of it is just setting standard parameters. So for practical use you can push most of the detail into a separate library file called `geminiENGINE319.py` and provided smaller programs in `gemini02.py` and `gemini03.py` for experimentation.

2. Download `gemini02.py`, `gemini03.py` and `geminiENGINE319.py` from Moodle.

3. Try running `gemini02.py`

Try entering the names of colours: Red, Green, Pale pink.

You will find that your first response is slow, but later ones are better.

What results do you get?

Now try entering sentences that might be suggestive of colour: Coral reef at midday.

Does this seem promising for lighting control? Look at the code – see what sense you can make of it. In particular, look at the instructions provided for the AI model. Try changing the instructions.

Be prepared to discuss.

In normal operation a program like ChatGPT will take answers to earlier questions into account when answering new ones. This is managed by using the whole history of a conversation when answering new questions. The program you have just tried does not maintain any history – each

question is answered as though it were the start of a new conversation. The library file provided can maintain history. It provides a different method for asking questions while maintaining history.

4. Try running `gemini03.py`

Demonstrate a series of inputs which take advantage of the history.

Hint: ask for a colour and then ask for a change to that colour.

Check that `gemini02.py` gives different results.

`geminiENGINEE319.py` – a small library to hide details of using gemini

```
# To run this code you need to install the following dependencies:
# pip install google-genai

from google import genai
from google.genai import types

client =
genai.Client(api_key="AIzaSyBzV_CqzrgUy024cNdCW0Um3RPMTdwarZk")

model = "gemini-2.5-flash"

config = types.GenerateContentConfig(
    temperature=0,
    thinking_config = types.ThinkingConfig(thinking_budget=-1),
    safety_settings=[
        types.SafetySetting(
            category="HARM_CATEGORY_HARASSMENT",
            threshold="BLOCK_LOW_AND_ABOVE", # Block most
        ),
        types.SafetySetting(
            category="HARM_CATEGORY_HATE_SPEECH",
            threshold="BLOCK_LOW_AND_ABOVE", # Block most
        ),
        types.SafetySetting(
            category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
            threshold="BLOCK_LOW_AND_ABOVE", # Block most
        ),
        types.SafetySetting(
            category="HARM_CATEGORY_DANGEROUS_CONTENT",
            threshold="BLOCK_LOW_AND_ABOVE", # Block most
        )
    ],
    tools=[types.Tool(googleSearch=types.GoogleSearch())])

def set_instructions(instructions):
    config.system_instruction = [types.Part.from_text(text=instructions)]

history = []

def ask_gemini(input_text):
    history = [(types.Content(role="user",
    parts=[types.Part.from_text(text=input_text)]))]
    response = client.models.generate_content(model=model,
    contents=history, config=config)
```

```

    return response.text

def ask_gemini_keep_history(input_text):
    history.append(types.Content(role="user",
    parts=[types.Part.from_text(text=input_text)]))
    response = client.models.generate_content(model=model,
    contents=history, config=config)
    history.append(types.Content(role="model",
    parts=[types.Part.from_text(text=response.text)]))
    return response.text

```

Gemini02.py – asking for colours with no conversation history

```

from geminiENGINE319 import set_instructions, ask_gemini

my_instructions = """
I have a lamp which can be set by giving red green and light levels
as numbers between 0 and 255, where 0 is dark and 255 is brightest.
I will write descriptions of colours and you should respond in the
format \"100,100,100\" substituting appropriate values for
the numbers in my format. Don't add any extra text and output
only one line.
"""

set_instructions(my_instructions)
while True:
    # Read text from the keyboard
    my_input_text = input("You: ")
    if my_input_text == "exit":
        break

    # Send it to gemini and print the response
    answer = ask_gemini(my_input_text)
    print("Gemini: ", answer)

```

Gemini03.py – asking for colours with conversation history for context

```

from geminiENGINE319 import set_instructions, ask_gemini_keep_history

my_instructions = """
I have a lamp which can be set by giving red green and light levels
as numbers between 0 and 255, where 0 is dark and 255 is brightest.
I will write descriptions of colours and you should respond in the
format \"100,100,100\" substituting appropriate values for
the numbers in my format. Don't add any extra text
"""

set_instructions(my_instructions)
while True:
    # Read text from the keyboard
    my_input_text = input("You: ")
    if my_input_text == "exit":
        break

```

```
# Send it to gemini and print the response
answer = ask_gemini_keep_history(my_input_text)
print("Gemini: ", answer)
```

Task 6: Controlling the smart light.

The final goal for this week's lab session is to make a program to control the light – in particular to set its colour. A good test of any program is to set users (yourself, partner or member of another group) a well-defined task. For this exercise the task is to get the light to match the colour of a paper sample (supplied). You can try to do this in one or more of three ways.

The three control systems are:

- Using a graphic user interface on screen
- Using Google's speech recognition and respond to word in the result
- Using Sphinx key phrase recognition with suitable command phrases

Sample files are provided for each, with code to go some way towards light control. Each system has been implemented in a pair of python files. There is a main program, which controls the light and imports a 'user interface' from the other python file. The main program files are identical except for the line importing the user interface file. The user interface file implements its approach.

light05.py, light06.py and light07.py are the main programs.

light05GUI.py constructs a Tk/Tk graphic user interface.

light06GSR.py uses Google Speech recognition.

light07Sphinx.py uses the Sphinx recognition engine.

You can run them by using Python to run the main program. It imports the other file.

1. Select a system and discuss how you might implement a program.
2. Plan your program and try building it.
3. Demonstrate and discuss the strengths and weaknesses of your approach.
4. Sketch a diagram showing the communications involved in the implementation of your system.